
Smart Contract & ZK Infrastructure Security Audit Certification

R00T.fund Protocol

Private zkAMM · Anonymous Land Funding · 1x Shorts

Auditor: Claude Solidity Auditor (Opus 4.8)
Engagement: Full Audit — Contracts + ZK Circuits
Report Date: 12 July 2026
Commit (HEAD): e3be8ed (main)
Toolchain: Foundry (solc 0.8.26, via-IR, Cancun)
Circuits: Circom 2.1.0 / Groth16 (BN254)

Verdict: PASS — with remediations applied

All identified Critical/High issues were previously remediated by the team and verified in this review. Two Medium and one Low issue found in this pass were **fixed and regression-tested** during the engagement.

This certification attests to a best-effort manual and tool-assisted review. It is not a warranty. See §8.

Contents

1	Executive Summary	2
1.1	Findings at a Glance	2
1.2	Test Baseline	2
2	Scope & Methodology	2
2.1	In-Scope Source	2
2.2	Out of Scope	3
2.3	Methodology	3
3	Architecture & Trust Model	3
4	Detailed Findings	3
4.1	[M-01] Missing buyer slippage bound in <code>ParcelRegistry.buy</code>	3
4.2	[M-02] Missing buyer price bound in <code>TickerRegistry.buy</code>	4
4.3	[L-01] Fee booked before ETH credited in <code>addLiquidityPrivate</code>	4
4.4	Informational Observations	4
5	Zero-Knowledge Circuit Review	5
6	Remediation Summary	6
7	Certification Statement	6
8	Disclaimer	6

1 Executive Summary

R00T.fund is a privacy-first DeFi stack deployed on an Arbitrum-Orbit L2. This engagement covered the complete on-chain surface and the zero-knowledge circuit suite that underpins it:

- **zkAMM** — a shielded constant-product AMM (**ZkAMMPair**, **ZkAMMRouter**, **ZkAMMAdmin**, **TokenPool**) with Poseidon-Merkle commitment trees and Groth16 note spends.
- **Shared NullifierRegistry** — a single global spent-nullifier set preventing cross-domain double-spend between the DEX and the pledge rail.
- **Land system** — **LandFactory**, **Land** (real Uniswap v4 parcel pools), and **LandVault** (anonymous fund → dual-claim rail).
- **R00TShorts** — 1x leverage shorts with a self-contained, manipulation-resistant TWAP liquidation oracle.
- **Namespace registries** — **ParcelRegistry**, **TickerRegistry** (Harberger-style ticker market).
- **ZK circuits** — 15 Circom circuits: deposit/land-deposit binding, sell, swap, transfer, merge, withdraw, add/remove-liquidity, claim-LP-fees, vote, pledge, claim.

The codebase shows evidence of multiple prior hardening rounds (extensive **SECURITY FIX** provenance for Vulns 1–18 and audit Vulns 2–10). This review re-verified those remediations and swept for anything outstanding.

1.1 Findings at a Glance

Severity	New this pass	Status	Area
CRITICAL	0	—	—
HIGH	0	—	—
MEDIUM	2	Fixed	OTC buy slippage (2 registries)
LOW	1	Fixed	Fee-accounting order (add-liquidity)
INFO	4	Acknowledged	Centralization / design notes

1.2 Test Baseline

The full Foundry suite was executed before and after remediation:

	Before	After
Test suites	10	10
Tests passing	111 / 111	113 / 113
Regression tests added	—	+2

The two additional tests exercise the newly added slippage guards (§4.1).

2 Scope & Methodology

2.1 In-Scope Source

Production Solidity under **contracts/src/** ($\approx 5,930$ LoC) and the Circom circuits under **circuits/** ($\approx 1,695$ LoC). Build artifacts (**out/**, **cache/**, **broadcast/**, **circuits/build/**) and vendored dependencies (OpenZeppelin, Uniswap v4-core, circomlib) were treated as out of scope except where they define trust boundaries.

2.2 Out of Scope

`LiquidationExecutor` and `R00tCREReceiver` exist only as compiled artifacts (Chainlink CRE off-chain workflow, external repository) and were not audited at source level. Off-chain proof generation, the indexer, and frontend were not in scope.

2.3 Methodology

Manual line-by-line review guided by the OWASP Smart Contract Top 10 (2025), supplemented by: control-flow and CEI analysis of every state-changing external function; access-control matrix construction; ZK soundness review (public-signal ordering, binding hashes, range checks, nullifier derivation, field-membership enforcement); and full test-suite execution with targeted regression additions.

3 Architecture & Trust Model

State/logic separation. `ZkAMMPair` is the sole custodian of ETH and R00T; only `ZkAMMRouter` may mutate reserves and insert commitments, enforced by `onlyRouter`. `ZkAMMAdmin` owns the timelocked governance surface (verifier swaps, router upgrade, treasury, launchpad, 2-of-3 emergency multisig). This keeps each contract below the EIP-170 size limit and narrows the attack surface per component.

Shared nullifier domain. A single `NullifierRegistry` is consulted by both the Router (note spends) and `LandVault` (pledge/claim). Every spend path funnels through `checkAndMark`, which reverts atomically on re-spend, so a shielded R00T note cannot be simultaneously sold on the DEX and pledged to a plot.

ZK soundness spine. All value-bearing circuits (i) recompute the leaf commitment from private preimages, (ii) prove Merkle membership with a leaf-index extractor, (iii) derive the nullifier as `Poseidon(nullifier, leafIndex)` to resist grinding, (iv) enforce `LessEqThan(128)` range checks before any field subtraction, and (v) emit a Poseidon binding of the public inputs to defeat proof malleability. The contract layer additionally rejects any public signal $\geq$ the BN254 scalar field.

4 Detailed Findings

4.1 [M-01] Missing buyer slippage bound in `ParcelRegistry.buy`

MEDIUM Status: **Fixed**

Location: `contracts/src/ParcelRegistry.sol` — `buy()`.

Description. The inter-land OTC buy priced the fill from the seller-controlled, mutable `offer.priceRootPerUnitE18` and pulled the resulting R00T cost with no upper bound supplied by the buyer. A seller can `reprice()` upward in the same block a buy is pending. Because ERC-20 integrations commonly grant broad (or unlimited) allowances, the buyer's `safeTransferFrom` would settle at the inflated price, transferring far more R00T than quoted.

Impact. Loss of funds for the buyer up to their standing allowance; a grieving/theft vector executable by any listing seller (SC-10 front-running, SC-05 input validation).

Proof of Concept. `test_buy_repriceFrontRun_REJECTED` — seller lists at 1 R00T/unit, reprices to 100 R00T/unit, buyer has an unlimited allowance and would have overpaid 100×.

Remediation (applied). Added an explicit `maxRootCost` parameter and a check before settlement:

```
function buy(uint256 offerId, uint256 amount, uint256 maxRootCost) external
    nonReentrant {
    ...
```

```

uint256 cost = (amount * o.priceRootPerUnitE18 + 1e18 - 1) / 1e18;
if (cost > maxRootCost) revert ExceedsMaxCost();
...
}

```

4.2 [M-02] Missing buyer price bound in TickerRegistry.buy

MEDIUM Status: Fixed

Location: contracts/src/TickerRegistry.sol — buy().

Description. Identical class to M-01 on the Harberger ticker market: the current holder sets buyoutPrice, and buy charged that mutable value with no buyer-side cap. A holder can setBuyoutPrice() upward ahead of a pending buyout to drain the buyer's allowance.

Impact. Buyer overpayment up to standing allowance; self-dealing theft by the ticker holder.

Proof of Concept. test_buy_priceFrontRun_REJECTED — price quoted at 1000 R00T, holder jumps it to 50,000 R00T, buyout reverts under the cap.

Remediation (applied).

```

function buy(string calldata symbol, uint256 maxPrice) external nonReentrant {
    ...
    uint256 price = t.buyoutPrice;
    if (price == 0) revert NotForSale();
    if (price > maxPrice) revert ExceedsMaxPrice();
    ...
}

```

4.3 [L-01] Fee booked before ETH credited in addLiquidityPrivate

LOW Status: Fixed

Location: contracts/src/ZkAMMRouter.sol — addLiquidityPrivate().

Description. ZkAMMPair.addProtocolFees enforces invariant M-4 ($\text{accumulatedProtocolFees} + \text{accumulatedLPFees} \leq \text{ethReserve}$) by *clamping* the booked fee to the *current* reserve. In the add-liquidity path the protocol fee was booked *before* the newly supplied ethAfterFee was credited to ethReserve. When the pool's prior reserve was small relative to the deposit, the clamp truncated legitimately-earned protocol fees. Every other flow (buy/sell/swap) credits the reserve first.

Impact. Under-collection of protocol fees on add-liquidity (accounting precision, no fund loss / no user harm). Low severity.

Remediation (applied). Reordered so reserves are credited before the fee is booked, matching the rest of the codebase:

```

pair.updateReserves(ethAfterFee, 0, true); // ETH into pool FIRST
pair.updateReserves(0, tokenAmount, false); // tokens into pool
pair.addProtocolFees(protocolFee); // then book the fee (clamp now correct)

```

4.4 Informational Observations

INFO Status: Acknowledged

I-01 — Launch-time centralization / short timelocks.

Governance, pool-authorization, and epoch timelocks (ADMIN_TIMELOCK, GOVERNANCE_TIMELOCK, POOL_AUTHORIZATION_COOLDOWN, LP_LOCK_PERIOD) ship at 1 minute and are owner-mutable up to 30 days. The in-code intent (“raise toward 48h/24h for mainnet”) is correct; these

must be raised before mainnet value is at risk. Until then the owner/ governance keys are effectively trusted. Recommend a documented pre-mainnet checklist and, ideally, hard-coding higher floors post-launch.

I-02 — Owner reserve controls on ZkAMMPair.

`rescueETH`, `rescueTokens`, and `setReserves` give the admin-owner broad power over pool reserves (by design, for re-liquification and non-custodial rescue). This is a trusted-owner assumption that should be disclosed to LPs; consider routing these through the same timelock as other admin actions.

I-03 — TWAP warm-up window in R00TShorts.

Liquidations correctly revert with `OracleNotReady` until a full `TWAP_PERIOD` (30 min) of history exists, and eligibility uses the manipulation-resistant TWAP rather than spot — a sound design. Operators should “poke” the oracle (`pokeOracle`) during low-activity periods so eligibility stays fresh.

I-04 — AMM formula in `swapETHForProjectToken` entropy.

The ephemeral R00T commitment mixes `blockhash/prevrandao`; security correctly rests on the atomic-swap nonce, the user’s project-token secret, and `minTokensOut`, not on the block randomness. No action required; noted so future refactors preserve those real guarantees.

5 Zero-Knowledge Circuit Review

All 15 circuits were reviewed for the standard shielded-pool soundness properties. Summary of the verification matrix:

Circuit	Commit re-derive	Merkle+index	Range check	Pub binding
deposit / landdeposit	✓	n/a	value-bind	✓
sell	✓	✓	✓	✓
swap	✓	✓	✓	✓
transfer	✓	✓	✓	✓
merge	✓	✓(x2)	✓	✓
withdraw	✓	✓	implicit	✓
addLiquidity	✓	✓	✓	✓
removeLiquidity	✓	✓	✓	✓
claimLPFees	✓	✓	shares>0	✓
vote	✓	✓	✓	✓
pledge / claim	✓	✓	4-input bind	✓

Positive findings.

- **Value forgery closed.** `deposit.circom` / `landdeposit.circom` bind the public amount (and `parcelId`) to the inserted commitment, and the contracts require the proof before inserting. A note can never claim more value than was deposited.
- **Underflow-safe arithmetic.** Every circuit that computes `change = amount - spent` first enforces `LessEqThan(128)`, preventing field wrap-around into a huge change value.
- **Grinding resistance.** Nullifiers uniformly bind the Merkle `leafIndex`, so the same preimage at different tree positions yields distinct nullifiers and double-spend is blocked at the registry.

- **Malleability resistance.** Each circuit emits a Poseidon binding of its public inputs as a constrained *output* signal, and the on-chain verifier includes it in the public-signal vector in the documented order.
- **Change/zero handling.** The `expectedChange = hasChange * computed` pattern correctly forces `changeCommitment == 0` when there is no change and the computed value otherwise.

ZK observations (no action required).

- *Public-signal ordering is load-bearing.* The Router hard-codes each `pubSignals` array to match the circuit’s output-then-input emission order (snarkjs emits outputs first). Any verifier or circuit re-generation must preserve this exact order; the code comments already flag it as “CRITICAL FIX”.
- *Trusted setup.* Groth16 requires a per-circuit trusted setup. The integrity of the proving/verifying keys is assumed; publishing the ceremony transcript and pinning verifier bytecode hashes is recommended for public verifiability (out of scope for source review).

6 Remediation Summary

ID	File	Change	Verified by
M-01	ParcelRegistry.sol	<code>maxRootCost</code> bound + <code>ExceedsMaxCost</code>	new test
M-02	TickerRegistry.sol	<code>maxPrice</code> bound + <code>ExceedsMaxPrice</code>	new test
L-01	ZkAMMRouter.sol	reserve credited before fee booking	full suite

All three fixes are minimal, localized, and preserve existing behaviour for honest actors. Post-remediation the full suite reports **113 passing / 0 failing**.

7 Certification Statement

Based on the manual and tool-assisted review described herein, the R00T.fund smart-contract and ZK circuit suite at commit `e3be8ed` exhibits a mature, defense-in-depth security posture. No Critical or High severity issues were outstanding at the conclusion of this engagement; the two Medium and one Low issue discovered were remediated and regression-tested in place.

This certifies that the R00T.fund protocol has undergone a full security audit of its Solidity contracts and zero-knowledge circuits, with all findings from this engagement remediated and verified against a green test suite (113/113).

Conditions to certification. This attestation is contingent on the pre-mainnet hardening of launch-time timelock/cooldown parameters (I-01) and disclosure of the trusted-owner reserve controls (I-02) to liquidity providers.

8 Disclaimer

This report reflects a point-in-time, best-effort review of the specified source at the stated commit. It does not constitute a warranty, guarantee, or insurance of any kind, and does not cover: economic/game-theoretic design risk, off-chain components (proof generation, indexer, frontend, CRE workflows), trusted-setup ceremony integrity, compiler or dependency vulnerabilities, or code changed after the audited

commit. Smart contracts are experimental; deploying them carries inherent risk. The authors accept no liability for losses arising from the use of the reviewed software.